

1. Difference between using an empty dependency array ([]) and omitting the dependency array altogether

- Empty array (useEffect(..., [])): Runs only once, after the first render (similar to componentDidMount). React does not re-run this effect unless the component is remounted.
- No array (useEffect(...)): Runs after every render.

Without dependencies, React re-executes the effect after every render, which can lead to performance issues.

2. Handling cleanup in useEffect and why it is necessary:

- Cleanup is used to undo side effects like event listeners, intervals, or subscriptions.
- It is necessary to prevent memory leaks and unwanted background behavior when a component unmounts.
- Example:

```
useEffect(() => {  
  const timer = setInterval(() => console.log("Running..."), 1000);  
  return () => clearInterval(timer); // Cleanup when component unmounts  
}, []);
```

3. Common pitfalls or mistakes to avoid when using useEffect:

- Forgetting dependency array → Effect runs on every render.
- Missing dependencies → Effect may use outdated data.
- Updating state without condition → Infinite re-renders.
- Ignoring cleanup → Memory leaks or background tasks continue after unmount.

4. Conditionally running effects in useEffect:

- Based on dependencies: useEffect(() => {...}, [variable]) — runs only when 'variable' changes.
- Based on condition inside effect:

```
useEffect(() => {  
  if (count > 5) {  
    console.log("Count greater than 5");  
  }  
}, [count]);
```